

# NMAP DEEP-DIVE

## PENTESTER FLAG-BY-FLAG NOTES

Scan types • Host discovery • Timing • Version/OS detection • NSE scripting • Port targeting • Real workflows

### AUTHORIZATION FIRST

Nmap sends real packets to real hosts. Only run any command in this document against systems and ranges you are explicitly authorized to test.

Nmap ("Network Mapper") discovers hosts and enumerates their services, versions, operating systems, and more — the gold-standard first tool in almost every engagement. This document walks through every flag covered in the lecture, in the order a real scan workflow actually uses them, with theory, exact syntax, and drills.

**Legend:** teal box = tip/best-practice · amber box = scenario/consideration · red box = warning/gotcha.

## SECTION 1

# Scan Techniques — -sS / -sT / -sU

How Nmap actually talks to a port

## -sS

Scan type

TCP SYN scan — the default scan technique

Also called the **stealth scan**. Nmap sends a SYN packet and reads the reply (SYN/ACK = open, RST = closed) but never completes the full TCP three-way handshake — no ACK is sent back, so many older logging mechanisms never record a full connection.

**SYN → SYN/ACK (open) or RST (closed) → Nmap logs result, connection never completed**

### Why it's the default

It's fast (half the packets of a full connect) and historically quieter in logs. Modern IDS/IPS now commonly detect SYN-scan patterns anyway, so "stealth" is relative today — but it's still the sensible default for speed.

### Practical command

```
nmap -sS 192.168.56.20
# Capitalisation matters: -sS, not -ss
```

**Use when:** Your default first scan on almost every engagement — fast, reasonably quiet, and requires raw-socket privileges (root/sudo on Linux).

## -sT

Scan type

TCP Connect scan — the full handshake

Completes the full TCP three-way handshake (SYN → SYN/ACK → ACK) then tears the connection down with RST. Because the OS's own network stack does the connecting (not raw sockets), it doesn't require elevated privileges — but it's noisier and slower.

**SYN → SYN/ACK → ACK (full connection made) → RST (teardown)**

### Practical command

```
nmap -sT 192.168.56.20
```

**Use when:** You lack raw-socket privileges, or you need higher assurance that a port is genuinely open — e.g. verifying results after a security control looked like it was manipulating SYN-scan responses.

## -sU

Scan type

UDP scan

Scans UDP ports instead of TCP. UDP has no handshake, so Nmap must send a probe and interpret silence, an ICMP port-unreachable, or a protocol-specific reply — which makes UDP scanning fundamentally slower. Expect the default UDP port set to take roughly 20 minutes to over an hour, even in a small lab.

#### Practical command

```
nmap -sU 192.168.56.20  
# Combine with a smaller port set to cut down the runtime:  
nmap -sU -p 53,67,123,161,500 192.168.56.20
```

#### When to reach for it

Fire it off early in the engagement if you have the time budget — kick it off in the background while you do other manual work. Many testers only run it when a TCP/SYN scan hasn't turned up enough to go on.

## SECTION 2

# Host Discovery Control — -Pn / -n

Skipping the parts of Nmap's order-of-operations you don't need

## -Pn

Discovery control

Treat all hosts as online — skip host discovery

By default, before scanning ports Nmap first verifies a host is up (ping-equivalent checks) and does name resolution. That verification step costs time. -Pn tells Nmap: "assume the target is alive, skip straight to port scanning."

**Normal order** → **Verify host is up** → **Name resolution** → **Port scan**

**With -Pn** → **Port scan (discovery skipped)**

### Practical command

```
nmap -Pn 192.168.56.20
```

**Use when:** You already confirmed the host is alive (e.g. from a previous scan or client-provided asset list) — used in nearly every follow-up scan in a real engagement.

## -n

Discovery control

No DNS/name resolution

Skips reverse-DNS lookups for each target, which otherwise add latency per host. Commonly paired with -Pn to strip out every non-essential pre-scan step.

### Practical command — the common combo

```
sudo nmap -Pn -n -T4 192.168.56.20
```

### Building your default workflow

Most working pentesters default to -Pn -n on nearly every scan once they know a host is up, purely for speed — then only re-enable discovery/name-resolution when there's a specific reason to (e.g. confirming liveness on an unknown range).

## SECTION 3

# Timing Templates — -T0 through -T5

The throttle: how fast, how loud, how detectable

Timing templates bundle together a whole set of underlying settings (parallelism, delays, timeouts) into one simple dial. Higher number = faster and louder; lower number = slower and quieter.

| Flag | Name       | Behaviour                                                            |
|------|------------|----------------------------------------------------------------------|
| -T0  | Paranoid   | Extremely slow — one probe roughly every 5 minutes; heaviest evasion |
| -T1  | Sneaky     | Very slow — built for IDS/IPS evasion                                |
| -T2  | Polite     | Slower than default — eases load on the network/target               |
| -T3  | Normal     | Default speed — balanced                                             |
| -T4  | Aggressive | Faster — assumes a reasonably fast, reliable network                 |
| -T5  | Insane     | Fastest — sacrifices accuracy for speed, easy to detect/trip alarms  |

### Practical commands

```
nmap -T4 192.168.56.20 # fast lab / internal engagement
nmap -T1 192.168.56.20 # red-team style, trying to fly under the radar
```

### Two different reasons to throttle down

- 1) **Bandwidth courtesy** — don't saturate a client's shared/slow network link.
- 2) **Detection avoidance** — many IDS/IPS flag the rapid-fire packet bursts that fast scans produce; slowing down helps a true red-team/adversary-emulation engagement stay under those thresholds.

### Go deeper

Run `man nmap` and search for "timing" — it documents every underlying value (in seconds/ms) that each -T level actually sets, plus manual overrides like `--scan-delay` and `--max-parallelism` for fully custom throttling.

## SECTION 4

# Service & Version Detection — -sV

"If you're not using -sV, you're doing yourself a disservice."

## -sV

Enumeration

Probe open ports for service + version

A port number alone is a guess ("port 80 is probably HTTP") — an administrator can put any service on any port. -sV actively probes each open port and reports the actual service and, where possible, its exact version string.

### Practical command

```
nmap -sV 192.168.56.20
# Example real output line:
# 80/tcp open  http Apache httpd 2.4.29
```

### Why this matters more than any other flag here

Version strings are exactly what you feed into a CVE/exploit-DB lookup. Skipping -sV means you're guessing at attack surface instead of knowing it.

### Chained into a real workflow

```
nmap -sV -Pn -n -T4 192.168.56.20
```

If a port returns no banner at all (as happened with an unidentified service in the lecture demo), Nmap will mark it unrecognised — that's your cue to manually enumerate it (netcat, curl, browser) rather than trust an automated guess.

### Manual follow-up on an unidentified port

```
nc -nv 192.168.56.20 3000
curl -sv http://192.168.56.20:3000
```

## SECTION 5

# OS Detection — -O

Fingerprinting the operating system, not just the services

### -O

Enumeration

OS detection (capital O, not zero)

Analyzes subtle differences in how a target's TCP/IP stack responds to craft a best-guess of the OS and kernel version. Accuracy improves significantly if Nmap can see at least one clearly open AND one clearly closed port on the target.

#### Practical command

```
sudo nmap -O -T5 -n -Pn 192.168.56.20
```

#### Improve your OS-detection accuracy

Deliberately include a port you already know is closed alongside your target's open ports. Nmap's OS fingerprinting algorithm uses the open/closed contrast to narrow down its guess — scanning only open ports gives it less to work with.

#### It's a best guess, not a certainty

Nmap will often say something like "OS detection performed, please report..." when confidence is low. Treat OS guesses the same way you treat any scanner finding: evidence to verify, not a fact to report as-is.

## SECTION 6

# NSE Scripting — --script

Nmap's built-in scripting engine — including light vuln checks

## --script

Automation

Invoke one or more Nmap Scripting Engine (NSE) scripts

NSE scripts live under a scripts directory (commonly `/usr/share/nmap/scripts` on Linux) and cover everything from banner grabbing to WHOIS lookups to vulnerability checks. You invoke them by name (with or without the `.nse` extension) via `--script`.

### Practical commands

```
ls /usr/share/nmap/scripts/ | less # see what's available
```

```
nmap --script=http-title 192.168.56.20
```

```
nmap --script=banner 192.168.56.20
```

```
nmap --script vuln 192.168.56.20 # run the whole 'vuln' category
```

| Common script/category | What it does                                                      |
|------------------------|-------------------------------------------------------------------|
| vuln                   | Category of scripts checking for a range of known vulnerabilities |
| http-title             | Grabs the of a web page on the target                             |
| banner                 | Grabs service banners on open ports                               |
| smb-os-discovery       | Extracts OS/host info via SMB                                     |
| ftp-anon               | Checks for anonymous FTP login allowed                            |

**Use when:** Targeted follow-up once `-sV` has told you what's running — pick a script relevant to that specific service instead of guessing.

## SECTION 7

# The All-In-One Flag — -A

Version detection + OS detection + default scripts + traceroute

## -A

Combo flag

Aggressive/all-in-one enumeration

-A bundles four things into one flag: service/version detection, OS detection, NSE's default script set, and traceroute. It's a fast way to get maximum information — but running it against every port on a large range is heavy.

### Practical command — the recommended workflow

# Step 1: find open ports fast

```
nmap -Pn -n -T4 -p- 192.168.56.20 -oG - | grep open
```

# Step 2: re-scan ONLY those open ports with -A for full depth

```
sudo nmap -A -T4 -n -Pn -p 22,80,443,8080 192.168.56.20
```

### Why two passes beats one

Running -A against all 65535 ports at once (open and closed) is slow and mostly wasted effort on closed ports. Find the open ports first with a lighter scan, then re-scan just those ports with -A for the deep dive. Faster, and you only get the info you actually need.

## SECTION 8

# Port Specification — -p

Nmap's default only checks the top 1000 — make sure that's enough

## -p

Scope control

Specify ports or port ranges

Without -p, Nmap scans its default "top 1000" most-common ports — not ports 0–1000, but the 1000 most frequently seen services, a list Nmap's maintainers periodically update. That default is a good start but will miss anything running on an unusual port.

| Syntax         | Meaning                                                 |
|----------------|---------------------------------------------------------|
| -p 80          | Scan a single port                                      |
| -p 80,443,8080 | Scan a comma-separated list of specific ports           |
| -p 8000-9000   | Scan an inclusive range                                 |
| -p-            | Scan all 65,535 ports (shorthand for -p 1-65535)        |
| -p U:53,T:80   | Mix protocols — UDP port 53 and TCP port 80 in one scan |

### Practical commands

```
nmap -p 80,3000,8080,8800,8899 192.168.56.20
nmap -p 8899-88999 192.168.56.20 # note: cap at 65535 in practice
nmap -p- 192.168.56.20 # every port, every time, when thorough scope allows
```

### The professional default

Run a full -p- pass at least once per host during a real engagement. The default top-1000 is a great fast first look, but it is not a substitute for a full port sweep before you sign off on "we checked everything."

# Putting It Together — A Real Workflow

How these flags chain into an actual engagement

## 1. The recommended step order

- 1 **Full port sweep, fast** — find out what's open across the whole range.
- 2 **Version detection on just the open ports** — know exactly what's running.
- 3 **OS detection** if it matters for your engagement.
- 4 **-A or targeted NSE scripts** on the ports/services that look interesting.
- 5 **UDP scan** in the background/early if time allows, or as a fallback when TCP alone didn't give you enough.

### Example chained commands

```
# 1. Fast full port sweep
nmap -Pn -n -T4 -p- 192.168.56.20 -oN full_sweep.txt

# 2. Version detection on the open ports found above (e.g. 22,80,443,8080)
nmap -sV -Pn -n -T4 -p 22,80,443,8080 192.168.56.20 -oN versions.txt

# 3. OS detection
sudo nmap -O -Pn -n -T4 192.168.56.20

# 4. Deep dive with -A on just the interesting ports
sudo nmap -A -Pn -n -T4 -p 22,80,443,8080 192.168.56.20 -oN deepdive.txt
```

## 2. Output flags worth always using

| Flag         | Saves as                                       |
|--------------|------------------------------------------------|
| -oN file.txt | Normal human-readable output                   |
| -oX file.xml | XML — feeds into other tools/report generators |
| -oA basename | All formats at once (normal + XML + grepable)  |

## 3. Quick-reference: every flag in this document

| Flag      | One-line meaning                                         |
|-----------|----------------------------------------------------------|
| -sS       | SYN/stealth scan (default) — half-open, fast             |
| -sT       | Full TCP connect scan — noisier, more certain            |
| -sU       | UDP scan — slow, often overlooked                        |
| -Pn       | Skip host-discovery, assume target is up                 |
| -n        | Skip DNS/name resolution                                 |
| -T0...-T5 | Timing template — paranoid (slowest) to insane (fastest) |

| Flag            | One-line meaning                                  |
|-----------------|---------------------------------------------------|
| -sV             | Service + version detection on open ports         |
| -O              | OS detection                                      |
| --script        | Run one or more NSE scripts (incl. vuln category) |
| -A              | Combo: -sV + -O + default scripts + traceroute    |
| -p              | Specify ports/ranges; -p- = all 65535             |
| -oN / -oX / -oA | Save output — normal / XML / all formats          |

### Go to the primary source

man nmap is genuinely worth reading start to finish once. Fyodor's (the creator's) own book and the documentation at [nmap.org/insecure.org](http://nmap.org/insecure.org) cover every flag here in far more depth, including the exact underlying timing values.

---

*Personal training reference — pair with the companion "Junior Pentester Field Notes" and "Pentester Tool Cheat Sheet" documents for the surrounding methodology.*